

Lappeenranta teknillinen yliopisto
LUT School of Engineering Sciences

Software Development Skills Front-End, Online course

Nguyen Lam Duc Anh, 002476548

LEARNING DIARY, Front-End Module

Link to course coding exercises repositories: [alexanh-webd/CoursePracticeSDSFe: Course practice coding exercises for SDS Front End](#)

Link to course project repository: [alexanh-webd/SDSFrontEnd_Project: Project for SDS Front End](#)

Link to video: [Course Project \(Software Development Skills Front-End, Online course\)](#)

Learning Diary: Introduction and Base HTML Module.

Date: 19 May 2026

I checked the course content and the requirements to pass the course. Since I have learned advanced web programming, this course is not difficult for me. I checked the video tutorial and saw what I was going to do. I started to do the first part of the video tutorial and understood the basics of HTML. I had also set up VS Code and other extensions before, including HTML, CSS, JavaScript, and Live Server to host the website from my local computer to the browser.

In the first part of the video, I learned how to set up the project. This includes the creation of HTML, CSS, JS folders, and index.html, which is the main HTML file for the homepage. Next, I created the nav bar like in the video. This took me a while to understand how to organize the items (elements) inside a nav bar. All the elements of the nav bar are linked to the main page (index.html). The menu is pretty interesting since I learned a new tag called ul (unordered list). Inside the tag, there are several elements (list items).

Today learning was fun as I have learned more about , tag, and the crucial part is the procedure of creating a nav bar. Nested elements is quite hard to understand at first when I started to study web programming. However, once I have understood it, everything

is not that hard to overcome. I just need to dive deeper into the core knowledge to fully understand it which takes quite much time as I have not expected.

Learning Diary: Links and Core CSS

Date: 20 May 2026

In the previous part, I created a nav bar. However, it was a boring nav bar with no styles or effects in it. That was the motivation for me to learn more about CSS to make the nav bar more appealing and attract more attention from the user. The first thing that I had to do was reset the margin and padding to 0 since there may be some elements with a default margin different from 0. This will cause minor issues later when the project gets larger and I need to style more elements. Next, I set the font-family that I was going to use for the whole page, the font-size, and the line-height. I also set the background color to white. I set the style for the link tag `<a>`, the `<ul>` tag, and also the `<img>` tag, which sets the max-width to 100% for the whole page.

After setting all the properties of the tags, I set the styles for individual classes. The first class is the container class (`.container`). I set the max-width to 1100px and aligned it to the middle of the homepage. Later, I styled the nav bar (`.navbar`) by setting the background color to white and adding general padding (top, bottom, left, and right) of 20px on each side.

Now, I had finished the styles for each container, which includes the logo and the navigation element in the `<ul>` tag. Now I wanted them to be in the same row and push the two containers to the left and right. This can be done by using `justify-content: space-between`.

Next, I modified the `<ul>` so that each `<li>` element within the `<ul>` tag would be displayed horizontally in the desktop view.

After this section, I learned more about the procedure of styling elements in CSS. The first thing that caught my attention was resetting all the properties by setting the margin and padding to 0. Next, I learned about nesting elements inside a bigger class. An example of a nesting component is the unordered list `<ul>` tag, which has `<li>` elements inside it. This can also be seen in the container class. A nav bar contains several containers. Each container contains images, links, unordered lists, etc. Another interesting thing is setting variables for colors since we do not want to type the color hex value many times. Instead, we can create a variable and use it immediately.

Learning Module: Buttons & Utility Classes

Date: 21 May 2026

In the previous part, I learned how to initialize CSS by setting padding and margin to 0. I learned how to set general variables to reuse multiple times during the development of the project, and I understood that one of the most important things in styling and HTML is nesting elements. Now, I will learn more about the properties of buttons and how to style them. The first thing that I had to do was set the display to inline-block, padding (top, bottom, left, and right), button color, font-weight (bold text), border-radius for the rounded

corners of the button, cursor to pointer, and a transition of 0.5s. The transition is quite interesting since if I hover the cursor over the button, the opacity of the button becomes lighter.

I also set the general style for other buttons that I am going to use in this project. By doing this, I can set the button style and color immediately during the development phase.

After this section, I learned how to style buttons and add effects for the cursor and the text opacity of the button. By doing this, I can create a cool transition effect when I hover the cursor over the button and the opacity changes. Another cool thing that I learned is creating general styles for other buttons (primary button, dark button, and block button), which allows me to develop the web page faster. It is like creating a ready-made styled button that is ready to use immediately.

Learning Module: CSS Grid & Cards

Date: 22 May 2026

Today, I made the card display for the comments. When I first thought about the cards, I was scared that they would be hard to implement. However, when I followed the tutorial, it was simply a div whose class name is "card". The card style can be modified, including the border-radius to make it look fancier, padding, background color of the card, and text color.

After creating the card, we had to place the cards in order. This can be done easily with display: grid. By setting the grid-template-columns to 1fr for each card, they are placed evenly across the web page. However, the gap between each card was 0, so they stuck to each other. By setting the gap to 30px, we could separate the cards easily.

I also started to do the project based on what I had learned. First, I thought about the topic that I wanted to do. I really love Gunpla model kits from Bandai. I kept the nav bar as it was. However, I changed the logo to another one from Bandai. I also wanted to finish the first part of the web page. I changed the background to a picture. Above that picture, I put a header and detailed information about what the web page is about. I also put a picture next to the text so that the web page looks more attractive. This can be done by changing the background-size to cover. For the picture to float above the background, I looked it up on Google, and it suggested changing the background-size to contain. I modified the size, padding, and the link to the background picture.

This section helped me learn more about CSS Grid, which is a powerful tool to align cards beautifully within the homepage. The card can be styled by setting border-radius, background color, text color, and grid-template, which allows me to put the card in the desired position.

Learning Module: FAQ elements

Date: 23 May 2026

For today's schedule, I wanted to finish the FAQ part from the tutorial video, which includes HTML, CSS, and a little bit of JavaScript.

The FAQ can be easily done by creating an unordered list within the section whose class name is “faq bg-light”. Within that section, I created a big div called “container-sm”. This div contains a header for the questions and an unordered list for the navigation bar. However, this navigation bar is not functional since I have not added any JavaScript yet. Next, I also added another div called “faq-content”. Within this faq-content div, I have several faq-groups, which are the questions posted on the webpage. These question cards contain an <h4> header to highlight the question posted by the user. Under the group header for the question is the group for the answer to the question.

Now I wanted to add more questions to the FAQ UI, and I could do that by copying the <div> header and <div> group for the answer.

Next, I added some styling to the FAQ. For the FAQ section, I added some padding. For the faq-group, I set the border-bottom and padding-bottom so that each question and answer is separated from the others. For the faq-group-header, I set its position to relative so that I could place it exactly where I wanted. I also styled the question header (<h4>). I modified the font-weight by trying different values that matched my preference. For the <i> tag, I styled its position to absolute, set the font-size, changed the cursor to pointer when hovered over, and adjusted the distance from the top, left, and right.

For the faq-menu, which is the navigation bar of the FAQ part, I set the display to flex so that the elements would be displayed on the same line. Next, I set the max-width, margin, justify-content to space-between, and aligned them to the center of the div.

Now, the next part is to add some functionality to the question group. I want it to be clickable and expanded to give me the answer. It is time for some JavaScript. There are two ways to implement this. The first is to add an event listener to each question and it will be expanded as we click on it. The other option is more efficient as we just need to add event listener to the parent div and it will figure out which child are we currently clicking on and expanding it automatically.

By adding an event listener to the parent div and a variable “e” (notation for event) as an input for the arrow function, and within that function, we add another event listener for the faq container. By testing several times and console log the e.target, we will know exactly which child are we clicking on. Modifying the code a bit more, we can log the header class name easily. Once the correct header is selected, I stored the parent FAQ group, the body section, and the icon into separate variables so they can be manipulated easily.

Next, I used classList.toggle() to dynamically switch the plus and minus icons whenever the user clicks on a question. I also toggled the visibility of the answer section by adding or removing the open class. Finally, I added functionality to close any other open FAQ sections whenever a new one is clicked. This improves the user experience because only one answer remains open at a time. To achieve this, I looped through all FAQ groups and removed the open class and reset the icons for the non-selected items. Overall, this approach made the FAQ section more interactive while keeping the JavaScript concise and scalable.

After this section, I found out that making the FAQ functioning is not an easy task. Maybe I have to learn more as it is important to my future career. I also learned more about the creation and styling of the FAQ box. It is pretty similar to what I have done so far in the previous part. Only the JavaScript part was new to me. The event listener is similar to what I have learned in Android development. To conclude, this is the hardest part to learn during the development of this project.

Learning Module: Mobile Menu & Responsiveness

Date: 24 May 2026

The journey is about to end as this will be the second last module that I have to learn so far. As I have done a lot of work for the webpage in the Desktop view, let move on to something new which will be view on a mobile phone. The motivation to learn this module not only because it is compulsory as a course work, but also because it is an important features and a must-have if I want to develop a web page. Since more people are using mobile devices, it is crucial to develop and arrange the display of the elements within the web page so that it can be compatible with many devices.

Let see what do I have to modify. It seems like I have to fix the hero group, and the nav bar. The FAQ looks good. The video looks good also. However, the comment section which is below the video needs to be modified.

I added the @media queries. What it does is to apply the CSS rules for a specific screen sizes. This allows us to apply specific CSS styling for the mobile device and other CSS rules for desktop screen. By using the max-width property, I can create rules for the screen size. This means that if the screen size is under 960px (max-width: 960px), we can apply display: none for example. I have tested it and it works.

Next I added a hamburger menu. This is sufficient for small screen to display the menu. I styled this hamburger button by adding to it width, height, background color, and margin so that it looks nicer.

Adding more details to the menu hamburger, it is time to implement the functionality for that button. Now I want the menu to visible as I click on the hamburger button and turn back to normal as I move away. This can be done by setting the another active class for the button. This class will set the right position properties of the menu to be 0. With a little bit more of Javascript, finally I can make it work. At first I got some errors and I realized that because I missed the active class. That is why when I hover over the hamburger menu, it does not show up.

The hardest part is over. Now it is time to fix the rest. I fix the hero display so that now it will show the background that is made for mobile devices. Style the background a little bit more by adding background-size, background-position, and height, the background now looks better than ever. I also fixed the hero content. This can be done by setting the text-align to be centered and finally stack all of the elements in the hero section on each other (display: block).

Testimonials can be modified so that it looks better in a mobile devices. Setting the max-width to 100% and the text-align to be centered, this allows the header of the testimonials to span across the screen and align in the center of the screen. Finally, change the grid-template-column to 1fr so that each box now stack on each other.

After this section, I learned more about @media queries and how to apply specific CSS styles to fit the screen sizes of many devices. The hardest part is figuring out where to place the elements on a mobile device so that it does not cause any confusion for the user since the screen is much smaller compared to a desktop. There is limited space to show all the information. I also noticed that changing just small properties, such as displaying elements so that they stack on top of each other, is sufficient to show information clearly on a mobile screen.

Learning Module: Website Deployment

Date: 35 May 2026

The course practice code is going to end with this final section. A web page is not alive if it is not deployed to the internet where people can use it instead of using live server. After learning this module, I now know how to deploy a simple website with html, CSS and JS.

The first thing that I have to add is the git ignore file. This file allows me to ignore node modules if I have when I am going to publish the code to GitHub as Node modules are large and may contains sensitive information. The next thing that I have to do is to create an empty repository in my GitHub. After that, git init, git add . , and git commit -m "Initial commit" to initialize the commit locally. git remote add origin <https://github.com/alexanh-webd/CoursePracticeSDSFe.git> to push the local commit to online GitHub.

At first, I got some problems pushing my local commit to my GitHub. This is because my github initial repository has a README file which is not created locally. That is why I got a lot of errors during this phase. However, if I pull origin main --allow-unrelated-histories to merge the two commit and publish it again, I can push it successfully.

Now I want to have https:// domain. First, I have to install gh-pages which allows me to deploy my webpage with a https:// domain. Adding homepage property to be <https://alexanh-webd.github.io/CoursePracticeSDSFe> to package.json file and deploy it, now for the first time, my web page is alive with a real https:// domain.

Learning Diary: Course Project

Date: 19-25 May 2026

I started my project and did the course exercises at the same time. Since 19 May 2026, I have already chosen the webpage that I want to implement. It will be a Gunpla webpage which is also my hobby. In the same day, I have done the HTML and CSS for the hero section and the nav bar. Not like in course exercises, the background is a bit boring, so I added my own

background with a picture of the universe, changed the introduction text, added the link to the button and the styling of each elements to fit the hero section.

On 20 May 2026, I started implementing the Details section for my webpage. I got a little bit stuck and problem when styling the picture alternatively to be positioned left and right. But thanks to Googling and asking from my friend, I was able to do it successfully. By changing the display to be flex, the content and the image will be at the same line. Adding some spacing, padding, the detail section now looks very appealing. Once I have done one div successfully, the other div are the same. I just need to download the image from Pinterest and search for the details of each product.

On 21 May 2026, my goal is to finish Comments, Pricing, and the frequently asked questions (FAQ). As it is similar to the sample project, I just need to modify the content so that it matches the spirit of Gunpla. The background of the pricing section is a bit boring at first, so I added another universe picture to make it look more appealing. I got a little bit of trouble also as the background is quite dark and the text of the Simple section is black initially. The text just disappeared as I proceeded. A simple solution to solve this problem is to change the background color of the simple section to be black which is similar to the Premium section. This can be done easily by changing the class name of the div since I have defined the css style for the black background already.

On 22 May 2026, I finalize my project and add the footer. I also added another cool function called “fade-up”. This allows the information to appear more professionally (floating from bottom to top), and I am very happy with the result. Finally, I published the project to GitHub and deploy the website. It went smoothly as I expected.